Supplementary Material: From Feed to Video: Coordinating Multi-Scenario Recommendation with Stackelberg Games

Wenjing Li *, Jiang Rong †, Zhenzhe Zheng *, Fan Wu *
*Shanghai Jiao Tong University
Shanghai, China
†Xiaohongshu
Beijing, China
{wenjingli, zhengzhenzhe}@sjtu.edu.cn, fwu@cs.sjtu.edu.cn
rongjiang@xiaohongshu.com

***Abstract*—This supplement documents an implementation snapshot associated with the Feed–Video recommendation study. It provides source-code provenance, configuration inventories, input schemas, verification commands, and detailed reproducibility gaps. The model interfaces and methodological implementation details are presented in the main paper. The package contains code and documentation only; it contains no datasets or model checkpoints. The archived scripts have unresolved execution and evaluation differences that are described explicitly. Consequently, the package should be read as an inspectable implementation snapshot, not as a verified reproduction of the manuscript's industrial results. The main paper remains a separate 12-page document followed by its references, with its theoretical proofs retained in the main text.**

I. Artifact Scope and Organization

The package includes twelve original Python files in `code/legacy/`. They cover the three Stackelberg optimizer variants, joint and hierarchical baselines, the causal-inference baseline, model definitions, JEV+ evaluation, confidence-interval utilities, ablations, and ablation plotting. The files are unchanged copies of the recovered project sources. The companion `docs/PROVENANCE.txt` records source-relative locations and SHA-256 hashes.

The historical project README contains the same Feed CTR improvements for SGD, Mirror Descent, and Bi-level optimization as the manuscript (20.00%, 7.59%, and 5.06%), and the same corresponding JEV+ improvements (19.74%, 17.90%, and 23.94%). This establishes a connection to the study, but does not identify the precise source revision, data split, or random seeds that produced each table. No industrial experiment was rerun while assembling this supplement.

The shared `models.py` was recovered from the parent project directory because the smaller code bundle imports it without including it. The other parent-directory additions are the causal-inference, ablation, and plotting scripts. Copying these files does not resolve every missing dependency; the remaining issues are listed in Section IV.

No raw, sampled, anonymized, or synthetic dataset files are distributed. The package also excludes private logs, checkpoints, pilot output directories, and later rebuttal-result directories. Input field names are documented to explain the code interface. This supplement makes no commitment to release industrial data.

TABLE I
Included implementation artifacts. All filenames are relative to the legacy-code directory.

File	Role in the recovered project
`SGD.py`	Independent and Stackelberg SGD comparison
`Mirror_descent.py`	Adam-based comparison
`bi_level.py`	RMSprop-based comparison
`Base_MARL.py`	Joint Feed/Video training baseline
`Hierarchical_RL.py`	Hierarchical baseline
`Causal_Inference.py`	Causal-inference baseline
`models.py`	Shared model and dataset classes
`jev_computation.py`	Journey construction and JEV+ utilities
`jev_stackelberg_comparison.py`	JEV+ comparisons across variants
`confidence_intervals.py`	Bootstrap utilities
`ablation_study.py`	Incremental model-component analysis
`exp_plots.py`	Ablation plot from embedded aggregate values

II. Configuration Inventory and Input Schema

The main paper's Section VIII, "Implementation and Reproducibility," now contains the shared-model architecture, historical-feature inputs, training and ranking defaults, input-processing conventions, JEV+ implementation behavior, and scope of artifact verification. The following compact inventories retain the script-level settings and exact field names for readers inspecting the code. They describe the archived snapshot rather than a certified final experimental configuration.

The Bi-level epoch count is imported from the joint-baseline module. The JEV utilities use SGD, Adam, or RMSprop depending on the selected variant. Most scripts

TABLE II
ARCHIVED TRAINING DEFAULTS (EPOCHS / LEARNING RATE / BATCH SIZE) AND EVALUATION LIMITS; NOT CERTIFIED FINAL-RUN CONFIGURATIONS.

Script (optimizer)	**Defaults**	**Test users**
SGD (SGD)	5 / 0.01 / 64	10,000-ID range
Mirror Descent (Adam)	5 / 0.01 / 64	10,000-ID range
Bi-level (RMSprop)	100 / 0.01 / 64	10,000-ID range
Joint baseline (Adam)	100 / 0.001 / 64	Up to 10,000
Hierarchical (Adam)	100 / 0.001 / 256	10
Causal (Adam)	3 / 0.001 / 256	10
Ablation (SGD)	5 / 0.01 / 64	Script-specific
JEV utilities (varied)	3 / 0.01 / 64	Caller-dependent

TABLE III
DOCUMENTED INPUT FIELDS; NO RECORDS ARE INCLUDED.

Input	Fields observed in the file header
Feed	`userid`, `noteid`, `click`, `feedpagetime`, `like`, `collect`, `share`, `follow`, `comment`
Video	`userid`, `noteid`, `reqid`, `videoplaytime`, `like`, `collect`, `comment`, `share`, `follow`, `dtm`, `hh`

expect `sampled_feed_page_data.csv` and `sampled_video_page_data.csv` in the working directory; the hierarchical and causal-inference scripts instead use paths relative to its parent. No input files are distributed. The main paper explains the split, chronology, journey-linkage, and metric differences that affect the interpretation of this interface.

III. ENVIRONMENT AND LIMITED VERIFICATION

The local environment inspected during packaging uses Python 3.9.6, PyTorch 2.7.1, NumPy 2.0.2, pandas 2.3.1, scikit-learn 1.6.1, Matplotlib 3.9.4, and SciPy 1.13.1. The included requirements file records these versions. It is not a recovered lockfile for the original experiment runs, and it does not capture production hardware or serving infrastructure.

The newly written verification utility parses the Python sources, checks the archived hashes, identifies missing local imports, and checks that no dataset or checkpoint files are present. It does not import training entry points. Its optional runtime mode exercises the shared model interfaces using small in-memory tensors and checks selected JEV branches using a unit-check fixture. It writes no dataset and performs no model training.

From the supplement directory, the syntax and integrity checks can be run with:

```
python code/verify_artifacts.py
```

After installing the declared dependencies, the optional module checks can be run with:

```
python code/verify_artifacts.py -runtime
```

The report included at `verification/checks.txt` records the checks actually performed. Runtime checks cover the historical-feature model paths; they do not validate every fallback branch, train a ranking model, reproduce confidence intervals, or reproduce an online experiment.

IV. KNOWN REPRODUCIBILITY GAPS

A. Execution and Dependency Gaps

The JEV comparison imports `models.train_model`, which is absent from the recovered shared model file. Mirror Descent and Bi-level scripts import `generate_confidence_report` and `compare_metrics_with_ci`, which are absent from the included confidence-interval utility. Many scripts perform data loading and training at top level. Restoring a missing function by guessing an implementation could change the experiment, so the original files have been retained and the missing interfaces documented.

B. Evaluation Semantics

The bootstrap utility samples user IDs with replacement and then stores them as dictionary keys. Repeated selections are collapsed, so sample multiplicity is not preserved. Some ranking-metric implementations construct the ideal NDCG list by sorting the retrieved top-k relevance values rather than deriving it from the full relevant candidate population. Together with the split and JEV differences described in the main paper, these observations prevent certification of the paper's confidence intervals or benchmark values from the recovered snapshot alone.

C. Model Branches and Experiment Provenance

When historical features are absent, the shared Video model constructs a new linear layer inside its forward method. The checked historical-feature paths avoid that branch, and passing their interface checks does not establish correctness of other model paths. The archived settings also differ in test-user limits, training epochs, scoring bonuses, and feature use. A confirmed final experiment revision is needed to determine which of these paths produced the submitted results.

No final-table run manifest, complete seed list, saved model state, timing environment, or online A/B assignment and logging artifact is included. The ablation plotting script contains aggregate percentages in its source. Recreating its plot would reproduce a visualization of those values, not the experiment that generated them. Existing pilot and rebuttal outputs were not substituted for the main-paper results.

V. RELATIONSHIP TO THE MAIN SUBMISSION

Section VIII of the main paper now contains the implementation and verification explanations. This supplement retains inventories, commands, and detailed limitations. It adds no empirical claims or appendix to the main document. Complete reproduction still requires the final experiment revision, missing helpers, and configuration, with any corrections accompanied by validation. All distributed files remain free of dataset records.